

CI/CD PIPELINE USING JENKINS AND DOCKER

Prepared by Arnav Soni for the Codegrid React/Vite project.

Docker Hub account: ArnavSoni2007

Submission summary

This report applies the supplied Docker assessment workflow to the existing Codegrid React/Vite application. The repository now contains a production multi-stage Dockerfile, an Nginx configuration, a Jenkins declarative pipeline, a Docker build-context policy, and complete operator instructions. The pipeline checks out the repository, installs and builds the application, verifies Docker, builds and smoke-tests the image, authenticates with a Jenkins-managed credential, and pushes numbered and latest tags to Docker Hub.

Docker image repository names must be lowercase. Therefore the Docker Hub account shown as Arnav-Soni2007 is referenced by the image name `arnavsoni2007/codegrid`.

Verification statement: The clean Vite production build was executed successfully in the supplied workspace. Docker 29.7.2 is installed, but the current Linux account cannot access `/var/run/docker.sock` and passwordless sudo is unavailable. Consequently, a live image build and Docker Hub push could not be truthfully recorded in this environment. The Jenkins pipeline is complete and ready to execute on a Jenkins agent with Docker permission and the `dockerhub-cred` credential.

Aim

To implement a Continuous Integration and Continuous Deployment pipeline with Jenkins and Docker for the Codegrid web application. The pipeline automatically builds the React application, packages it in a small Nginx runtime image, verifies the container, and publishes the image to Docker Hub.

Tools Used

- Jenkins: automation server for the CI/CD pipeline.
- Docker: image building and container execution.
- Docker Hub: registry for versioned and latest images.
- GitHub: source repository used by the Jenkins checkout stage.
- Node.js and npm: deterministic dependency installation and Vite build.
- React and Vite: frontend application framework and build tool.
- Nginx: production static-file server and SPA fallback.

System Requirements

- Linux or Windows host with Git, Node.js, npm, and Docker installed.
- Jenkins with Pipeline, Git, and Credentials Binding plugins.
- Jenkins service account permitted to use the Docker daemon.
- GitHub repository containing the Codegrid project and Jenkinsfile.
- Docker Hub account ArnavSoni2007 and a scoped access token.

Project Structure

```
project/
|-- src/
|   |-- main.jsx      React application
|   |-- styles.css    Responsive UI styling
|-- index.html        Vite entry document
|-- package.json      npm scripts and dependencies
|-- package-lock.json reproducible dependency lock
|-- Dockerfile        multi-stage production image
```

```

|-- nginx.conf          Nginx SPA/static configuration
|-- .dockerignore      reduced Docker build context
|-- Jenkinsfile         complete CI/CD pipeline
|-- README.md          setup and run instructions
'-- submission/
    '-- docker_cicd_assignment.pdf

```

Dockerfile Configuration

The first stage uses Node.js only to install dependencies and generate the optimized Vite distribution. The second stage copies only the generated files into Nginx. Development dependencies and source tooling are absent from the runtime layer.

```

FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY index.html ./
COPY src ./src
RUN npm run build

FROM nginx:1.27-alpine AS runtime
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=3s \
  --start-period=5s --retries=3 \
  CMD wget --quiet --tries=1 --spider \
    http://127.0.0.1/ || exit 1
CMD ["nginx", "-g", "daemon off;"]

```

The Nginx configuration provides history-route fallback through index.html, long-lived caching for fingerprinted assets, and basic response security headers. The .dockerignore file excludes Git metadata, node_modules, build output, environment files, reports, and the supplied assessment PDF from the image context.

Jenkins Configuration

Step 1 - Install required plugins

Open Manage Jenkins, then Plugins. Install the Git, Pipeline, and Credentials Binding plugins. Restart Jenkins if requested. Docker must be installed on the agent and the Jenkins service account must be allowed to access the Docker daemon.

Step 2 - Add Docker Hub credentials

Open Manage Jenkins, Credentials, System, Global credentials, then Add Credentials. Select Username with password. Use ArnavSoni2007 as the username and a Docker Hub access token as the password. Set the credential ID to dockerhub-cred. The token must never be committed to the repository.

Step 3 - Create the pipeline job

Choose New Item, enter a job name such as codegrid-docker-cicd, select Pipeline, and choose Pipeline script from SCM. Select Git, enter the GitHub repository URL, set the branch to */main, and retain Jenkinsfile as the script path.

Step 4 - Pipeline stages

1. Checkout Code uses checkout scm so the pipeline builds the exact configured revision.
2. Install and Build Application runs npm ci and npm run build.

3. Verify Docker prints the Docker version and fails early if Docker is unavailable.
4. Build Docker Image creates arnavsoni2007/codegrid with the Jenkins build number and latest tags.
5. Smoke Test Image starts the container and checks that the served HTML contains the Codegrid title.
6. Login and Push to Docker Hub injects dockerhub-cred, authenticates through standard input, and pushes both tags.

The smoke-test container is stopped in an always block, and docker logout is executed after every pipeline result. Concurrent executions are disabled to avoid test-container name collisions.

Core Jenkinsfile Logic

```
environment {
    DOCKER_USERNAME = 'arnavsoni2007'
    IMAGE_NAME = 'codegrid'
    IMAGE_TAG = "${BUILD_NUMBER}"
}

stage('Build Docker Image') {
    steps {
        sh '''
            docker build \
                --tag "$DOCKER_USERNAME/$IMAGE_NAME:$IMAGE_TAG" \
                --tag "$DOCKER_USERNAME/$IMAGE_NAME:latest" .
        '''
    }
}

stage('Login and Push to Docker Hub') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-cred',
            usernameVariable: 'DOCKERHUB_USER',
            passwordVariable: 'DOCKERHUB_TOKEN'
        )]) {
            sh '''
                printf '%s' "$DOCKERHUB_TOKEN" | docker login \
                    --username "$DOCKERHUB_USER" --password-stdin
                docker push "$DOCKER_USERNAME/$IMAGE_NAME:$IMAGE_TAG"
                docker push "$DOCKER_USERNAME/$IMAGE_NAME:latest"
            '''
        }
    }
}
```

Execution Procedure

1. Commit and push Dockerfile, nginx.conf, .dockerignore, Jenkinsfile, README.md, and the application files to the main branch.
2. Confirm that dockerhub-cred exists in Jenkins and contains the ArnavSoni2007 account plus an access token.
3. Open the codegrid-docker-cicd Jenkins job and select Build Now.
4. Watch Console Output. All six stages must be green before publication occurs.

5. Open Docker Hub and confirm both arnavsoni2007/codegrid:<build-number> and arnavsoni2007/codegrid:latest.
6. Optionally verify from any Docker host with docker pull arnavsoni2007/codegrid:latest and docker run --rm -p 8080:80 arnavsoni2007/codegrid:latest.

Verification Evidence

Application production build - passed

The build was executed in the supplied project on 4 September 2026.

```
$ npm run build
```

```
> codegrid@0.1.0 build
```

```
> vite build
```

```
vite v8.1.5 building client environment for production...
```

```
15 modules transformed.
```

```
dist/index.html 0.46 kB | gzip: 0.30 kB
```

```
dist/assets/index-zWEZDuK9.css 11.57 kB | gzip: 3.45 kB
```

```
dist/assets/index-CBuQmwvM.js 200.62 kB | gzip: 62.95 kB
```

```
built in 266ms
```

Docker availability - client passed, daemon access blocked

```
$ docker version
```

```
Client: Docker Engine - Community
```

```
Version: 29.7.2
```

```
permission denied while trying to connect to the Docker API at
unix:///var/run/docker.sock
```

```
$ ls -l /var/run/docker.sock
```

```
srw-rw---- 1 root docker ... /var/run/docker.sock
```

This is a host permission issue rather than a project build error: the current user is not a member of the docker group, and sudo requires an interactive password. A Jenkins agent must be granted Docker access before running the job. No live Docker image, registry push, or Jenkins SUCCESS result is claimed by this report.

Expected Successful Output on Jenkins

```
[Pipeline] stage (Checkout Code)          SUCCESS
```

```
[Pipeline] stage (Install and Build Application) SUCCESS
```

```
[Pipeline] stage (Verify Docker)          SUCCESS
```

```
[Pipeline] stage (Build Docker Image)      SUCCESS
```

```
[Pipeline] stage (Smoke Test Image)        SUCCESS
```

```
[Pipeline] stage (Login and Push to Docker Hub) SUCCESS
```

```
Published arnavsoni2007/codegrid:<build-number> and :latest
```

```
Finished: SUCCESS
```

Result

The CI/CD assignment has been implemented for the Codegrid React/Vite project. The repository contains all code and instructions required for Jenkins to perform deterministic application installation, frontend compilation, production image construction, container smoke testing, Docker Hub authentication, and publication of immutable numbered and latest tags.

Local frontend compilation passed. Final live publication is an operator step because it requires host Docker permission, a running Jenkins service, and the private dockerhub-cred value. Once those prerequisites are present, Build Now executes the full workflow without exposing the access token in source control.

Conclusion

This implementation demonstrates a secure and repeatable Jenkins and Docker workflow for a modern frontend application. The multi-stage Docker design separates compilation from runtime, Nginx serves only optimized artifacts, the pipeline tests the image before release, and Docker Hub credentials remain under Jenkins control. These choices improve repeatability, image size, credential hygiene, and release traceability.

Submission Checklist

- React/Vite application builds successfully: YES.
- Production Dockerfile supplied: YES.
- Nginx runtime and SPA configuration supplied: YES.
- Docker build context exclusions supplied: YES.
- Jenkins checkout/build/test/login/push pipeline supplied: YES.
- Docker Hub account configured as ArnavSoni2007: YES, normalized to lowercase in image references.
- Documentation and generated PDF supplied: YES.
- Live Docker daemon build and Docker Hub push verified here: NO - host permission and private credential boundary.